

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
"Высшая школа экономики"»

Московский институт электроники и математики им.
А.Н. Тихонова НИУ ВШЭ
Департамент компьютерной инженерии

Курс: Алгоритмизация и программирование

ОТЧЕТ
по лабораторной работе №4

Раздел	Максимальная оценка	Итоговая оценка
Работа программы	1	
Тесты	1	
Правильность алгоритма	3	
Листинг	0,5	
Ответы на вопросы	2	
Дополнительное задание	3	
Итого		

Студент: Солодилов Михаил Анатольевич
Группа: БИВ253
Вариант: 232 (5, 3, 2)
Руководитель:
Оценка:
Дата сдачи:

Оглавление

Задание	2
Внешняя спецификация	3
Листинг программы	4
Распечатка тестов к программе и результатов	7

Задание

Дано k литерных строк. Каждая строка содержит латинские и русские буквы, цифры, а также все возможные разделители. Требуется:

1. Выделить из каждой строки и напечатать подстроки, ограниченные с обеих сторон одной или несколькими точками.
2. Среди выделенных подстрок найти подстроку (если таких подстрок несколько, выбирается первая из них), содержащую цифры и круглые скобки.
3. Преобразовать исходную строку, которой принадлежит найденная подстрока, следующим образом, удалить путем сдвига все символы, отличные от русских букв.

Внешняя спецификация

Лабораторная работа №4

Задание №1

Введите k от 1 до $ARRAY_SIZE$:
< k >

★

До $1 \leq k \leq ARRAY_SIZE$

Введите << k >> строк длины не более $STRING_SIZE$:
< $S[0]$ >, < $S[1]$ >, ..., < $S[k-1]$ >

При $n = 0$

Не нашлось подстрок, ограниченных точками

Иначе

Нашлось << n >> подстрок, ограниченных точками:
<< $substrs[0]$ >>, << $substrs[1]$ >>, ..., << $substrs[n-1]$ >>

При $found.size = 0$

Не нашлось строки со скобками и цифрами

Иначе

Нашлась строка со скобками и цифрами: "<< $found$ >>"
После очистки букв не из русского алфавита: "<< $found$ >>"

Листинг программы

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 constexpr size_t STRING_SIZE = 1024;
6 constexpr size_t ARRAY_SIZE = 1024;
7
8 typedef struct String
9 {
10     const char* data;
11     size_t size;
12 } String;
13
14 typedef struct String_array
15 {
16     String data[ARRAY_SIZE];
17     size_t size;
18 } String_array;
19
20 static String_array
21 split_string_array_by_dots(size_t k,
22                             const char strings[ARRAY_SIZE][STRING_SIZE]);
23 static String_array split_string_by_dots(const char* string);
24
25 static String find_correct_substring(const String_array* subtrs);
26 static bool check_correct_string(String string);
27
28 static void filter_non_cyrillic(String* string);
29
30 static void print_string(String str);
31 static size_t input_k(void);
32 static void clean_buffer(void);
33
34 int main()
35 {
36     size_t k = input_k();
37
38     printf("Введите %zu строк длины не более %zu:\n", k, STRING_SIZE - 1);
39     char strings[ARRAY_SIZE][STRING_SIZE] = {};
40
41     for (size_t i = 0; i < k; i++)
42     {
43         fgets(strings[i], STRING_SIZE, stdin);
44     }
45
46     String_array subtrs = split_string_array_by_dots(k, strings);
47     if (subtrs.size)
48     {
49         printf("Нашлось %zu подстрок, ограниченных точками:\n", subtrs.size);
50         for (size_t i = 0; i < subtrs.size; i++)
51         {
52             print_string(subtrs.data[i]);
53             printf("\n");
54         }
55     }
56     else
57     {
58         printf("Не нашлось подстрок, ограниченных точками\n");
59     }
60
61     String found = find_correct_substring(&subtrs);
62
63     if (found.size)
64     {
65         printf("Нашлась строка со скобками и цифрами:\n\"");
66         print_string(found);
67         printf("\n");
68
69         filter_non_cyrillic(&found);
70         printf("После очистки букв не из русского алфавита:\n\"");
71         print_string(found);
72         printf("\n");
73     }
```

```

74     else
75     {
76         printf("Не нашлось строки со скобками и цифрами\n");
77     }
78 }
79
80 static String_array
81 split_string_array_by_dots(size_t k,
82                             const char strings[ARRAY_SIZE][STRING_SIZE])
83 {
84     String_array result = {};
85
86     for (size_t i = 0; i < k; i++)
87     {
88         String_array sub_res = split_string_by_dots(strings[i]);
89
90         for (size_t j = 0; j < sub_res.size; j++)
91         {
92             result.data[result.size++] = sub_res.data[j];
93         }
94     }
95
96     return result;
97 }
98
99 static String_array split_string_by_dots(const char* string)
100 {
101     String_array result = {};
102     size_t len = strlen(string);
103
104     const char* start = strchr(string, '.');
105     if (!start)
106     {
107         return result;
108     }
109
110     while (start < string + len)
111     {
112         while (start < string + len && *start == '.')
113         {
114             start++;
115         }
116         const char* end = strchr(start, '.');
117
118         if (!end)
119         {
120             break;
121         }
122
123         result.data[result.size++] = (String) {start, end - start};
124         start = end;
125     }
126
127     return result;
128 }
129
130 static String find_correct_substring(const String_array* subtrs)
131 {
132     for (size_t i = 0; i < subtrs->size; i++)
133     {
134         if (check_correct_string(subtrs->data[i]))
135         {
136             return subtrs->data[i];
137         }
138     }
139     return (String) {};
140 }
141
142 static bool check_correct_string(String string)
143 {
144     bool has_brackets = memchr(string.data, '(', string.size)
145                         && memchr(string.data, ')', string.size);
146     if (!has_brackets)
147     {
148         return false;
149     }

```

```

150
151     for (char digit = '0'; digit <= '9'; digit++)
152     {
153         if (memchr(string.data, digit, string.size))
154         {
155             return true;
156         }
157     }
158
159     return false;
160 }
161
162 static void filter_non_cyrillic(String* string)
163 {
164     const char* read = string->data;
165     char* write = (char*)read;
166     size_t new_size = string->size;
167
168     while (read < string->data + string->size)
169     {
170         if (*read & (1 << 8))
171         {
172             *(write++) = *read;
173         }
174         else
175         {
176             new_size--;
177         }
178         read++;
179     }
180     *write = '\0';
181     string->size = new_size;
182 }
183
184 static void print_string(String str)
185 {
186     for (size_t i = 0; i < str.size; i++)
187     {
188         fputc(str.data[i], stdout);
189     }
190 }
191
192 static void clean_buffer(void)
193 {
194     while (getchar() != '\n')
195         ;
196 }
197
198 static size_t input_k(void)
199 {
200     size_t k = 0;
201     do
202     {
203         printf("Введите k от 1 до %zu:\n", ARRAY_SIZE);
204         scanf("%zu", &k);
205         clean_buffer();
206     } while (!(1 <= k && k <= ARRAY_SIZE));
207
208     return k;
209 }

```

Распечатка тестов к программе и результатов

№	Исходные данные	Результат
1	$k = 1, S[0 : 0] = [\text{abba}]$	Не нашлось подстрок, ограниченных точками Не нашлось строки со скобками и цифрами
2	$k = 2, S[0 : 2] = [\text{y..b..c ..d, .dog при(1-2)ветfe.}]$	Нашлось 3 подстрок, ограниченных точками: b c dog при(1-2)ветfe Нашлась строка со скобками и цифрами: "dog при(1-2)ветfe" После очистки букв не из русского алфавита: "привет"
3	$k = 1, S[0 : 0] = [\text{.adas.dog приветfeline.}]$	adas dog приветfeline Не нашлось строки со скобками и цифрами